MySQL 基本 SQL 語法快速參考卡

📋 四大核心操作速查

操作	用途	基本語法	風險等級
SELECT	查詢資料	`SELECT * FROM table WHERE condition`	✅ 安全
INSERT	新增資料	`INSERT INTO table (col1, col2) VALUES (val1, val2)`	⚠️ 低風險
UPDATE	修改資料	`UPDATE table SET col1=val1 WHERE condition`	🔶 中風險
DELETE	刪除資料	`DELETE FROM table WHERE condition`	🔴 高風險

🔍 SELECT 查詢語法

完整語法結構

```
SELECT [DISTINCT] column1, column2, ...
FROM table_name
WHERE condition
GROUP BY column
HAVING group_condition
ORDER BY column [ASC|DESC]
LIMIT count [OFFSET skip];
```

常用查詢模式

基本查詢

```
SELECT * FROM customers;                    -- 查詢所有欄位
SELECT name, city FROM customers;           -- 查詢特定欄位
SELECT name AS '姓名' FROM customers;       -- 使用別名
```

條件查詢

```
WHERE age > 18                              -- 數值比較
WHERE name = '張三'                         -- 字串相等
WHERE name LIKE '張%'                       -- 模糊查詢 (張開頭)
WHERE name LIKE '%小%'                      -- 模糊查詢 (包含小)
WHERE age BETWEEN 18 AND 65                 -- 範圍查詢
WHERE city IN ('台北', '高雄')              -- 多值查詢
WHERE manager_id IS NULL                    -- 空值查詢
WHERE age > 18 AND city = '台北'            -- 且條件
WHERE city = '台北' OR city = '高雄'        -- 或條件
```

排序與限制

```
ORDER BY age ASC                          -- 升序（預設）
ORDER BY age DESC                         -- 降序
ORDER BY city, age                        -- 多欄位排序
LIMIT 10                                  -- 前10筆
LIMIT 10 OFFSET 5                        -- 跳過5筆，取10筆
```

分組統計

```
GROUP BY department                       -- 按部門分組
HAVING COUNT(*) > 5                      -- 分組後篩選

-- 常用聚合函數
COUNT(*)          -- 計數
SUM(salary)       -- 總和
AVG(salary)       -- 平均
MAX(salary)       -- 最大值
MIN(salary)       -- 最小值
```

➕ INSERT 插入語法

基本語法

```
INSERT INTO table_name (column1, column2) VALUES (value1, value2);
```

常用插入模式

單筆插入

```
-- 指定欄位插入
INSERT INTO customers (id, name, city) VALUES (1, '張三', '台北');

-- 全欄位插入（按表結構順序）
INSERT INTO customers VALUES (1, '張三', '台北', '台灣', NOW());
```

多筆插入

```
INSERT INTO customers (id, name, city) VALUES
    (1, '張三', '台北'),
    (2, '李四', '高雄'),
    (3, '王五', '台中');
```

從其他表插入

```
INSERT INTO customers_backup SELECT * FROM customers;
INSERT INTO summary (dept, count)
    SELECT department, COUNT(*) FROM employees GROUP BY department;
```

處理重複

```
INSERT IGNORE INTO customers VALUES (1, '張三', '台北');  -- 忽略重複
INSERT INTO customers VALUES (1, '新名字', '台北')
    ON DUPLICATE KEY UPDATE name = VALUES(name);          -- 重複時更新
REPLACE INTO customers VALUES (1, '替換', '台北');       -- 替換
```

🔄 UPDATE 更新語法

基本語法

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

⚠ **重要警告**: 更新前務必備份，並確保 WHERE 條件正確！

常用更新模式

基本更新

```
UPDATE customers SET city = '新北' WHERE id = 1;           -- 單欄位
UPDATE customers SET city = '新北', country = '台灣' WHERE id = 1;  -- 多欄位
```

計算更新

```
UPDATE products SET price = price * 1.1;                   -- 全部漲價10%
UPDATE employees SET salary = salary + 5000 WHERE dept = 'IT';  -- 加薪
UPDATE products SET stock = stock - 1 WHERE id = 100;      -- 減庫存
```

條件更新

```
UPDATE employees SET salary =
    CASE
        WHEN dept = 'IT' THEN salary * 1.1
        WHEN dept = 'Sales' THEN salary * 1.05
        ELSE salary
    END;
```

安全更新流程

```
-- 1. 先查詢要更新的資料
SELECT * FROM customers WHERE city = '台北';

-- 2. 開啟事務
START TRANSACTION;

-- 3. 執行更新
UPDATE customers SET city = '新北' WHERE city = '台北';

-- 4. 檢查結果
SELECT * FROM customers WHERE city = '新北';

-- 5. 確認後提交 (或 ROLLBACK 回滾)
COMMIT;
```

❌ DELETE 刪除語法

基本語法

```
DELETE FROM table_name WHERE condition;
```

🚨 **危險警告**: 刪除操作不可恢復，務必謹慎操作！

常用刪除模式

條件刪除

```
DELETE FROM customers WHERE id = 1;                    -- 刪除特定記錄
DELETE FROM orders WHERE order_date < '2023-01-01';    -- 刪除舊資料
DELETE FROM products WHERE stock = 0 AND discontinued = TRUE;  -- 複合條件
```

安全刪除流程

```sql
-- 1. 先查詢要刪除的資料
SELECT * FROM customers WHERE city = '測試';

-- 2. 計算影響筆數
SELECT COUNT(*) FROM customers WHERE city = '測試';

-- 3. 備份要刪除的資料 (可選)
CREATE TABLE deleted_customers AS
SELECT * FROM customers WHERE city = '測試';

-- 4. 執行刪除
DELETE FROM customers WHERE city = '測試';

-- 5. 驗證結果
SELECT COUNT(*) FROM customers WHERE city = '測試';
```

軟刪除 (推薦)

```sql
-- 添加刪除標記欄位
ALTER TABLE customers ADD COLUMN is_deleted BOOLEAN DEFAULT FALSE;

-- 軟刪除 (標記為已刪除)
UPDATE customers SET is_deleted = TRUE WHERE id = 1;

-- 查詢時排除已刪除
SELECT * FROM customers WHERE is_deleted = FALSE;
```

DELETE vs TRUNCATE

```sql
DELETE FROM table_name;        -- 逐筆刪除，可恢復，有日誌
TRUNCATE TABLE table_name;     -- 整表清空，快速，不可恢復
```

🛡️ 安全性與最佳實踐

查詢優化

```sql
-- ✅ 好的做法
SELECT id, name FROM customers WHERE id = 1;          -- 使用索引欄位
SELECT * FROM orders WHERE date >= '2024-01-01';      -- 避免函數條件
SELECT * FROM products ORDER BY price LIMIT 10;       -- 限制結果數量
```

```
-- ❌ 避免的做法
SELECT * FROM large_table;                              -- 避免全表掃描
SELECT * FROM orders WHERE YEAR(date) = 2024;          -- 避免函數條件
```

資料修改前的檢查

```
-- 檢查資料完整性
SELECT COUNT(*) FROM orders WHERE customer_id NOT IN
    (SELECT id FROM customers);

-- 檢查異常資料
SELECT * FROM products WHERE price <= 0;
SELECT * FROM employees WHERE salary IS NULL;
```

事務使用

```
START TRANSACTION;

    -- 執行相關的多個操作
    UPDATE accounts SET balance = balance - 100 WHERE id = 1;
    UPDATE accounts SET balance = balance + 100 WHERE id = 2;

    -- 檢查結果無誤後提交
COMMIT;  -- 或使用 ROLLBACK; 回滾
```

🔧 常用函數速查

字串函數

```
CONCAT(str1, str2)          -- 字串串接
SUBSTRING(str, start, len)   -- 取子字串
LENGTH(str)                 -- 字串長度
UPPER(str), LOWER(str)      -- 大小寫轉換
TRIM(str)                   -- 去除空格
```

數值函數

```
ROUND(num, decimals)        -- 四捨五入
CEILING(num)                -- 無條件進位
FLOOR(num)                  -- 無條件捨去
ABS(num)                    -- 絕對值
MOD(num1, num2)             -- 取餘數
```

日期函數

```
NOW()                        -- 現在時間
CURDATE()                    -- 現在日期
DATE_FORMAT(date, format)    -- 日期格式化
YEAR(date), MONTH(date)      -- 取年份、月份
DATE_ADD(date, INTERVAL n DAY)  -- 日期計算
```

條件函數

```
IF(condition, true_val, false_val)           -- 條件判斷
CASE WHEN condition THEN val1 ELSE val2 END  -- 多條件判斷
COALESCE(val1, val2, ...)                   -- 回傳第一個非NULL值
NULLIF(val1, val2)                          -- 相等時回傳NULL
```

🎯 實用 SQL 片段

資料探索

```
-- 查看表結構
DESCRIBE table_name;
SHOW CREATE TABLE table_name;

-- 查看資料分布
SELECT COUNT(*), MIN(date), MAX(date) FROM orders;
SELECT column, COUNT(*) FROM table GROUP BY column;
```

效能分析

```
-- 分析查詢執行計劃
EXPLAIN SELECT * FROM customers WHERE city = '台北';

-- 檢查索引使用
SHOW INDEX FROM table_name;
```

備份與恢復

```
-- 建立表備份
CREATE TABLE backup_table AS SELECT * FROM original_table;

-- 快速複製表結構
CREATE TABLE new_table LIKE existing_table;
```

📊 SQL 執行順序記憶法

```
🔢 記住執行順序: From Where Group Having Select Order Limit
   📝 記憶口訣：「From哪裡 Where什麼條件 Group分組 Having分組條件 Select選什麼 Order怎麼排 Limit取多少」
```

🆘 緊急救援指令

誤刪資料救援

```
-- 如果誤刪，立即停止所有操作
-- 1. 檢查是否有備份
SELECT * FROM backup_table_name;

-- 2. 檢查二進制日誌 (需要DBA協助)
SHOW BINARY LOGS;

-- 3. 使用事務回滾 (如果還在事務中)
ROLLBACK;
```

長時間執行的查詢

```
-- 查看正在執行的查詢
SHOW PROCESSLIST;

-- 終止特定查詢 (需要權限)
KILL QUERY process_id;
```

💡 建議將此參考卡片列印或儲存，隨時查閱使用！